TCC70xx MCU BSP

API Specification for Interrupt Controller

Rev. 0.10 [A]

2021-09-03

Preliminary version

※ The information in this document is subject to change without notice and should not be construed as a commitment by Telechips Inc.

Kindly visit www.telechips.com for more information.

TABLE OF CONTENTS

Contents

Figures

Tables

1 INTRODUCTION

This document describes the interrupt controller device driver (hereinafter referred to as GIC driver) of TCC70xx MCU BSP. For more detailed information of the interrupt controller in the TCC70xx MCU Sub-system, refer to Chapter 4 Interrupt Controller in *"TCC70xx Full Specification"* [1].

2 TERMS AND ABBREVIATIONS

Table 2.1 Terms and Abbreviation

GIC	Generic Interrupt Controller
ISR	Interrupt Service Routine

3 SOURCE FILES

3.1 Location of Source Files

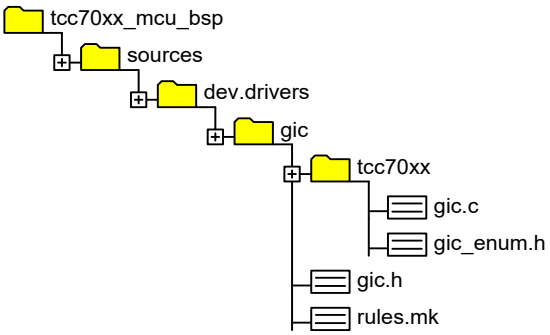

Figure 3.1 Location of Source Files

3.2 Simple Description of Source Files

- gic.c
  - GIC source
- gic_enum.h
  - Interrupt Index enumerations
- gic.h
  - GIC driver API header
- rules.mk
  - Rules used for build

4 ENUMERATIONS

4.1 MCU Shared Peripherals Interrupts

Shared Peripherals Interrupt (SPI) index of MCU peripherals

Declaration

```
enum
{
    GIC_CAN0_0                  = (GIC_SPI_START + 0UL),
    GIC_CAN0_1                  = (GIC_SPI_START + 1UL),
    GIC_CAN1_0                  = (GIC_SPI_START + 2UL),
    GIC_CAN1_1                  = (GIC_SPI_START + 3UL),
    GIC_CAN2_0                  = (GIC_SPI_START + 4UL),
    GIC_CAN2_1                  = (GIC_SPI_START + 5UL),
    GIC_GPSB                    = (GIC_SPI_START + 6UL),
    GIC_GPSB1                   = (GIC_SPI_START + 7UL),
    GIC_GPSB2                   = (GIC_SPI_START + 8UL),
    GIC_GPSB3                   = (GIC_SPI_START + 9UL),
    GIC_GPSB4                   = (GIC_SPI_START + 10UL),
    GIC_GPSB0_DMA               = (GIC_SPI_START + 11UL),
    GIC_GPSB1_DMA               = (GIC_SPI_START + 12UL),
    GIC_GPSB2_DMA               = (GIC_SPI_START + 13UL),
    GIC_GPSB3_DMA               = (GIC_SPI_START + 14UL),
    GIC_GPSB4_DMA               = (GIC_SPI_START + 15UL),
    GIC_UART0                   = (GIC_SPI_START + 16UL),
    GIC_UART1                   = (GIC_SPI_START + 17UL),
    GIC_UART2                   = (GIC_SPI_START + 18UL),
    GIC_UART3                   = (GIC_SPI_START + 19UL),
    GIC_UART4                   = (GIC_SPI_START + 20UL),
    GIC_UART5                   = (GIC_SPI_START + 21UL),
    GIC_DMA0                    = (GIC_SPI_START + 22UL),
    GIC_DMA1                    = (GIC_SPI_START + 23UL),
    GIC_DMA2                    = (GIC_SPI_START + 24UL),
    GIC_DMA3                    = (GIC_SPI_START + 25UL),
    GIC_DMA4                    = (GIC_SPI_START + 26UL),
    GIC_DMA5                    = (GIC_SPI_START + 27UL),
    GIC_DMA6                    = (GIC_SPI_START + 28UL),
    GIC_DMA7                    = (GIC_SPI_START + 29UL),
    GIC_I2C                     = (GIC_SPI_START + 30UL),
    GIC_I2C1                    = (GIC_SPI_START + 31UL),
    GIC_I2C2                    = (GIC_SPI_START + 32UL),
    GIC_I2C3                    = (GIC_SPI_START + 33UL),
    GIC_I2C4                    = (GIC_SPI_START + 34UL),
    GIC_I2C5                    = (GIC_SPI_START + 35UL),
    GIC_VS_I2C0                 = (GIC_SPI_START + 36UL),
    GIC_VS_I2C1                 = (GIC_SPI_START + 37UL),
    GIC_VS_I2C2                 = (GIC_SPI_START + 38UL),
    GIC_VS_I2C3                 = (GIC_SPI_START + 39UL),
    GIC_VS_I2C4                 = (GIC_SPI_START + 40UL),
    GIC_VS_I2C5                 = (GIC_SPI_START + 41UL),
    GIC_ICTC0                   = (GIC_SPI_START + 42UL),
    GIC_ICTC1                   = (GIC_SPI_START + 43UL),
    GIC_ICTC2                   = (GIC_SPI_START + 44UL),
    GIC_ICTC3                   = (GIC_SPI_START + 45UL),
```

```
    GIC_ICTC4                   = (GIC_SPI_START + 46UL),
    GIC_ICTC5                   = (GIC_SPI_START + 47UL),
    GIC_RED_ICTC0               = (GIC_SPI_START + 48UL),
    GIC_RED_ICTC1               = (GIC_SPI_START + 49UL),
    GIC_RED_ICTC2               = (GIC_SPI_START + 50UL),
    GIC_RED_ICTC3               = (GIC_SPI_START + 51UL),
    GIC_RED_ICTC4               = (GIC_SPI_START + 52UL),
    GIC_RED_ICTC5               = (GIC_SPI_START + 53UL),
    GIC_ADC0                    = (GIC_SPI_START + 54UL),
    GIC_ADC1                    = (GIC_SPI_START + 55UL),
    GIC_TIMER_0                 = (GIC_SPI_START + 56UL),
    GIC_TIMER_1                 = (GIC_SPI_START + 57UL),
    GIC_TIMER_2                 = (GIC_SPI_START + 58UL),
    GIC_TIMER_3                 = (GIC_SPI_START + 59UL),
    GIC_TIMER_4                 = (GIC_SPI_START + 60UL),
    GIC_TIMER_5                 = (GIC_SPI_START + 61UL),
    GIC_TIMER_6                 = (GIC_SPI_START + 62UL),
    GIC_TIMER_7                 = (GIC_SPI_START + 63UL),
    GIC_TIMER_8                 = (GIC_SPI_START + 64UL),
    GIC_TIMER_9                 = (GIC_SPI_START + 65UL),
    GIC_WATCHDOG                = (GIC_SPI_START + 66UL),
    GIC_PMU_WATCHDOG            = (GIC_SPI_START + 67UL),
    GIC_DEFAULT_SLV_ERR         = (GIC_SPI_START + 68UL),
    GIC_SFMC                    = (GIC_SPI_START + 69UL),
    GIC_CR5_PMU                 = (GIC_SPI_START + 70UL),
    GIC_EXT0                    = (GIC_SPI_START + 71UL),
    GIC_EXT1                    = (GIC_SPI_START + 72UL),
    GIC_EXT2                    = (GIC_SPI_START + 73UL),
    GIC_EXT3                    = (GIC_SPI_START + 74UL),
    GIC_EXT4                    = (GIC_SPI_START + 75UL),
    GIC_EXT5                    = (GIC_SPI_START + 76UL),
    GIC_EXT6                    = (GIC_SPI_START + 77UL),
    GIC_EXT7                    = (GIC_SPI_START + 78UL),
    GIC_EXT8                    = (GIC_SPI_START + 79UL),
    GIC_EXT9                    = (GIC_SPI_START + 80UL),
    GIC_EXTn0                   = (GIC_SPI_START + 81UL),
    GIC_EXTn1                   = (GIC_SPI_START + 82UL),
    GIC_EXTn2                   = (GIC_SPI_START + 83UL),
    GIC_EXTn3                   = (GIC_SPI_START + 84UL),
    GIC_EXTn4                   = (GIC_SPI_START + 85UL),
    GIC_EXTn5                   = (GIC_SPI_START + 86UL),
    GIC_EXTn6                   = (GIC_SPI_START + 87UL),
    GIC_EXTn7                   = (GIC_SPI_START + 88UL),
    GIC_EXTn8                   = (GIC_SPI_START + 89UL),
    GIC_EXTn9                   = (GIC_SPI_START + 90UL),
    GIC_FMU_IRQ                 = (GIC_SPI_START + 91UL),
    GIC_FMU_FIQ                 = (GIC_SPI_START + 92UL),
    GIC_I2S_DMA                 = (GIC_SPI_START + 93UL),
    GIC_GMAC                    = (GIC_SPI_START + 94UL),
    GIC_GMAC_TX0                = (GIC_SPI_START + 95UL),
    GIC_GMAC_TX1                = (GIC_SPI_START + 96UL),
    GIC_GMAC_RX0                = (GIC_SPI_START + 97UL),
    GIC_GMAC_RX1                = (GIC_SPI_START + 98UL),
    GIC_SRAM_ECC                = (GIC_SPI_START + 99UL),
    GIC_DMAC_                   = (GIC_SPI_START + 100UL),
    GIC_DMAC_INT_CLEAR          = (GIC_SPI_START + 101UL),
    GIC_DMAC_INTERR             = (GIC_SPI_START + 102UL),
    GIC_HSM_WDT_OUTPUT          = (GIC_SPI_START + 103UL),
    GIC_MBOX_SLV_TX             = (GIC_SPI_START + 104UL),
```

```
    GIC_MBOX_MST_TX             = (GIC_SPI_START + 105UL),
    GIC_AES                     = (GIC_SPI_START + 106UL),
    GIC_TRNG                    = (GIC_SPI_START + 107UL),
    GIC_TRNG_FAD                = (GIC_SPI_START + 108UL),
    GIC_PKE                     = (GIC_SPI_START + 109UL),
    GIC_PKE_ECC_S               = (GIC_SPI_START + 110UL),
    GIC_PKE_ECC_D               = (GIC_SPI_START + 111UL),
    GIC_HSM_DEFAULT_SLV_ERR     = (GIC_SPI_START + 112UL),
    GIC_HSM_CPU_SYS_RESET_REQ   = (GIC_SPI_START + 113UL),
    GIC_PFLASH                  = (GIC_SPI_START + 114UL),
    GIC_DFLASH                  = (GIC_SPI_START + 115UL),
    GIC_RTC_ALARM               = (GIC_SPI_START + 116UL),
    GIC_RTC_WAKEUP              = (GIC_SPI_START + 117UL),
};
```

Description

Value	Description
GIC_CAN0_0	CAN FD Controller-0[0]
GIC_CAN0_1	CAN FD Controller-0[1]
GIC_CAN1_0	CAN FD Controller-1[0]
GIC_CAN1_1	CAN FD Controller-1[1]
GIC_CAN2_0	CAN FD Controller-2[0]
GIC_CAN2_1	CAN FD Controller-2[1]
GIC_GPSB	GPSB Controller-0
GIC_GPSB1	GPSB Controller-1
GIC_GPSB2	GPSB Controller-2
GIC_GPSB3	GPSB Controller-3
GIC_GPSB4	GPSB Controller-4
GIC_GPSB0_DMA	GPSB Controller-0 DMA
GIC_GPSB1_DMA	GPSB Controller-1 DMA
GIC_GPSB2_DMA	GPSB Controller-2 DMA
GIC_GPSB3_DMA	GPSB Controller-3 DMA
GIC_GPSB4_DMA	GPSB Controller-4 DMA
GIC_UART0	UART Controller-0
GIC_UART1	UART Controller-1
GIC_UART2	UART Controller-2
GIC_UART3	UART Controller-3

Value	Description
GIC_UART4	UART Controller-4
GIC_UART5	UART Controller-5
GIC_DMA0	DMA Controller-0
GIC_DMA1	DMA Controller-1
GIC_DMA2	DMA Controller-2
GIC_DMA3	DMA Controller-3
GIC_DMA4	DMA Controller-4
GIC_DMA5	DMA Controller-5
GIC_DMA6	DMA Controller-6
GIC_DMA7	DMA Controller-7
GIC_I2C	I2C Master Controller-0
GIC_I2C1	I2C Master Controller-1
GIC_I2C2	I2C Master Controller-2
GIC_I2C3	I2C Master Controller-3
GIC_I2C4	I2C Master Controller-4
GIC_I2C5	I2C Master Controller-5
GIC_VS_I2C0	I2C Virtual Slave-0
GIC_VS_I2C1	I2C Virtual Slave-1
GIC_VS_I2C2	I2C Virtual Slave-2
GIC_VS_I2C3	I2C Virtual Slave-3
GIC_VS_I2C4	I2C Virtual Slave-4
GIC_VS_I2C5	I2C Virtual Slave-5
GIC_ICTC0	ICTC-0
GIC_ICTC1	ICTC-1
GIC_ICTC2	ICTC-2
GIC_ICTC3	ICTC-3
GIC_ICTC4	ICTC-4
GIC_ICTC5	ICTC-5
GIC_RED_ICTC0	Redundancy ICTC-0

Value	Description
GIC_RED_ICTC1	Redundancy ICTC-1
GIC_RED_ICTC2	Redundancy ICTC-2
GIC_RED_ICTC3	Redundancy ICTC-3
GIC_RED_ICTC4	Redundancy ICTC-4
GIC_RED_ICTC5	Redundancy ICTC-5
GIC_ADC0	ADC Controller0
GIC_ADC1	ADC Controller1
GIC_TIMER_0	Timer-0
GIC_TIMER_1	Timer-1
GIC_TIMER_2	Timer-2
GIC_TIMER_3	Timer-3
GIC_TIMER_4	Timer-4
GIC_TIMER_5	Timer-5
GIC_TIMER_6	Timer-6
GIC_TIMER_7	Timer-7
GIC_TIMER_8	Timer-8
GIC_TIMER_9	Timer-9
GIC_WATCHDOG	Windowed Watchdog Timer
GIC_PMU_WATCHDOG	PMU Windowed Watchdog Timer
GIC_DEFAULT_SLV_ERR	Default Slave Error Interrupt Handler
GIC_SFMC	Serial-NOR Flash Memory Controller (SFMC)
GIC_CR5_PMU	Cortex-R5 Performance Monitoring Unit (PMU)
GIC_EXT0	External IRQ input-0
GIC_EXT1	External IRQ input-1
GIC_EXT2	External IRQ input-2
GIC_EXT3	External IRQ input-3
GIC_EXT4	External IRQ input-4
GIC_EXT5	External IRQ input-5
GIC_EXT6	External IRQ input-6

Value	*Description*
GIC_EXT7	*External IRQ input-7*
GIC_EXT8	*External IRQ input-8*
GIC_EXT9	*External IRQ input-9*
GIC_EXTn0	*External IRQn input-0*
GIC_EXTn1	*External IRQn input-1*
GIC_EXTn2	*External IRQn input-2*
GIC_EXTn3	*External IRQn input-3*
GIC_EXTn4	*External IRQn input-4*
GIC_EXTn5	*External IRQn input-5*
GIC_EXTn6	*External IRQn input-6*
GIC_EXTn7	*External IRQn input-7*
GIC_EXTn8	*External IRQn input-8*
GIC_EXTn9	*External IRQn input-9*
GIC_FMU_IRQ	*Fault Management Unit (FMU) IRQ*
GIC_FMU_FIQ	*Fault Management Unit (FMU) FIQ*
GIC_I2S_DMA	*I2S DMA*
GIC_GMAC	*GMAC Controller*
GIC_GMAC_TX0	*GMAC Tx0*
GIC_GMAC_TX1	*GMAC Tx1*
GIC_GMAC_RX0	*GMAC Rx0*
GIC_GMAC_RX1	*GMAC Rx1*
GIC_SRAM_ECC	*Cortex-M0 SRAM ECC*
GIC_DMAC	DMAC Interrupt
GIC_DMAC_INT_CLEAR	DMAC_INTTC (transfer completed)
GIC_DMAC_INTERR	DMAC_INTERR (transfer error)
GIC_HSM_WDT_OUTPUT	HSM Watchdog Timer Output
GIC_MBOX_SLV_TX	Mailbox Slave TX Interrupt
GIC_MBOX_MST_TX	Mailbox Master TX Interrupt
GIC_AES	AES Interrupt

Value	*Description*
GIC_TRNG	TRNG Interrupt
GIC_TRNG_FAD	TRNG Fault attack detection Interrupt
GIC_PKE	PKE Interrupt
GIC_PKE_ECC_S	PKE ECC Single-bit error correction Interrupt
GIC_PKE_ECC_D	PKE ECC Double-bit error detection Interrupt
GIC_HSM_DEFAULT_SLV_ERR	HSM Default Slave Interrupt
GIC_HSM_CPU_SYS_RESET_REQ	HSM_CPU_SYSRESET Request
GIC_PFLASH	Program eFlash
GIC_DFLASH	Data eFlash
GIC_RTC_ALARM	RTC Alarm
GIC_RTC_WAKEUP	*RTC Wakeup*

Remark

None

5 CONSTANTS

5.1 GIC Control Value

These constants are used to configure GIC's distributor and CPU interface.

Declaration

```
#define ARM_BIT_GIC_DIST_ICDDCR_EN   (0x00000001UL)
#define GIC_CPUIF_CTRL_ENABLEGRP0    (0x00000001UL)
#define GIC_CPUIF_CTRL_ENABLEGRP1    (0x00000002UL) //Currently not used
#define GIC_CPUIF_CTRL_ACKCTL        (0x00000004UL) //Currently not used
#define GIC_SGI_TO_TARGETLIST        (0UL)
```

Description

Value	Description
ARM_BIT_GIC_DIST_ICDDCR_EN	Global enable for forwarding pending Group 0 interrupts from the Distributor to the CPU interfaces.
GIC_CPUIF_CTRL_ENABLEGRP0	Enable for the signaling of Group 0 interrupts by the CPU interface to the connected processor.
GIC_CPUIF_CTRL_ENABLEGRP1	Enable for the signaling of Group 1 interrupts by the CPU interface to the connected processor (Currently not used) If you enable this function, you have to implement the relevant code.
GIC_CPUIF_CTRL_ACKCTL	Currently not used. ARM deprecates use of **GICC_CTLR.AckCtl**, and strongly recommends using a software model where **GICC_CTLR.AckCtl** is set to 0.
GIC_SGI_TO_TARGETLIST	Forward the SGI to the CPU interfaces specified in the CPUTargetList field of GICD_SGIR. In **GIC_IntSGI()** function, CPUTargetList field is set to CPU #0 because R5 is single core.

Remark

None

5.2 Interrupt Type Value

These constants are used to determine the type of interrupt trigger.

Declaration

```
#define GIC_INT_TYPE_LEVEL_HIGH      (0x1UL)
#define GIC_INT_TYPE_LEVEL_LOW       (0x2UL)
#define GIC_INT_TYPE_EDGE_RISING     (0x4UL)
#define GIC_INT_TYPE_EDGE_FALLING    (0x8UL)
#define GIC_INT_TYPE_EDGE_BOTH       (GIC_INT_TYPE_EDGE_RISING | GIC_INT_TYPE_EDGE_FALLING)

#define GIC_PRIORITY_NO_MEAN           (16UL)
```

Description

Value	Description
GIC_INT_TYPE_LEVEL_HIGH	level – sensitive
GIC_INT_TYPE_LEVEL_LOW	level - sensitive (unused define)
GIC_INT_TYPE_EDGE_RISING	edge-triggered
GIC_INT_TYPE_EDGE_FALLING	edge-triggered (unused define)
GIC_INT_TYPE_EDGE_BOTH	Only supported in external interrupt (GIC_EXT0 ~ GIC_EXT9). Refer to MCU Shared Peripherals Interrupts.
GIC_PRIORITY_NO_MEAN	Setting this value as a priority value does not change the priority value. In this case, the Interface function does not return an error. **Note:** The interrupt priority is initialized to 10 in GIC_Init, so it remains the same unless you change it.

Remark

None

5.3 Interrupt Index Value

These constants are used for reference in the GIC driver and the device drivers.

Declaration

```
#define GIC_PPI_START        (16UL)
#define GIC_SPI_START        (32UL)

#define GIC_EINT_START_INT   (GIC_EXT0)
#define GIC_EINT_END_INT     (GIC_EXT9)
#define GIC_EINT_NUM         (GIC_EXTn0 - GIC_EXT0)

#define GIC_INT_SRC_CNT      (GIC_RTC_WAKEUP)
```

Description

Value	Description
GIC_PPI_START	Private Peripherals Interrupt start index
GIC_SPI_START	Shared Peripherals Interrupt start index
GIC_EINT_START_INT	External Interrupt start index
GIC_EINT_END_INT	External Interrupt end index
GIC_EINT_NUM	External Interrupt total number
GIC_INT_SRC_CNT	Total count of Interrupt source

Remark

None

6 STRUCTURES

6.1 GICIntFuncPtr_t

The GIC Driver manages ISR callback function pointer for each interrupt index, and ***GICIntFuncPtr_t*** is a structure for the GIC's management of ISR callback function pointer.

Declaration

```
typedef struct GICIntFuncPtr
{
    GICIsrFunc ifpFunc;
    uint8      ifpIsBothEdge;
    void *     ifpArg;
} GICIntFuncPtr_t;
```

Description

Value	*Description*
ifpFunc	*Callback function for ISR. Refer to GICIsrFunc.*
ifpIsBothEdge	*If this value is true, it means that ifpFunc(ISR callback) is called on rising and falling edges of interrupt source. Both edges are only supported by external interrupt (GIC_EXT0 ~ GIC_EXT9).*
ifpArg	*ifpFunc input argument*

Remark

None

6.2 GICDistributor_t

The GIC's Distributor register is handled using the following structure:

Declaration

```
typedef struct GICDistributor
{
    uint32 dCTRL;
    uint32 dTYPER;
    uint32 dIIDR;
    uint32 dRSVD1[29];
    uint32 dIGROUPRn[32];
    uint32 dISENABLERn[32];
    uint32 dICENABLERn[32];
    uint32 dISPENDRn[32];
    uint32 dICPENDRn[32];
    uint32 dISACTIVERn[32];
    uint32 dICACTIVERn[32];
    uint32 dIPRIORITYRn[255];
    uint32 dRSVD3[1];
    uint32 dITARGETSRn[255];
    uint32 dRSVD4[1];
    uint32 dICFGRn[64];
    uint32 dPPISR[1];
    uint32 dSPISRn[15];
    uint32 dRSVD5[112];
    uint32 dSGIR;
    uint32 dRSVD6[3];
    uint32 dCPENDSGIRn[4];
    uint32 dSPENDSGIRn[4];
} GICDistributor_t;
```

Description

Value	Description
dCTRL	**GICD_CTRL**, Distributor Control Register
dTYPER	**GICD_TYPER**, Interrupt Controller Type Register
dIIDR	**GICD_IIDR**, Distributor Implementer Identification Register
dIGROUPRn	**GICD_IGROUPRn**, Interrupt Group Registers
dISENABLERn	**GICD_ISENABLERn**, Interrupt Set-Enable Registers
dICENABLERn	**GICD_ICENABLERn**, Interrupt Clear-Enable Registers
dISPENDRn	**GICD_ISPENDRn**, Interrupt Set-Pending Registers
dICPENDRn	**GICD_ICPENDRn**, Interrupt Clear-Pending Registers
dISACTIVERn	**GICD_ISACTIVERn**, Interrupt Set-Active Registers
dICACTIVERn	**GICD_ICACTIVERn**, Interrupt Clear-Active Registers

Value	*Description*
dIPRIORITYRn	***GICD_IPRIORITYRn**, Interrupt Priority Registers*
dITARGETSRn	***GICD_ITARGETSRn**, Interrupt Processor Targets Registers*
dICFGRn	***GICD_ICFGRn**, RW IMPLEMENTATION DEFINED Interrupt Configuration Registers*
dPPISR	***GICD_PPISR**, Private Peripheral Interrupt Status Register*
dSPISRn	***GICD_SPISRn**, Shared Peripheral Interrupt Status Registers*
dSGIR	***GICD_SGIR**, Software Generate Interrupt Register*
dCPENDSGIRn	***GICD_CPENDSGIRn**, SGInterrupt Clear-Active Registers*
dSPENDSGIRn	***GICD_SPENDSGIRn**, SGInterrupt Set-Active Registers*
dRSVD1, dRSVD3, dRSVD4, dRSVD5, dRSVD6	***Reserved***

Remark

None

6.3 GICCpuInterface_t

The GIC's CPU-interface register is handled using the following structure:

Declaration

```
Typedef struct GICCpuInterface
{
    uint32 cCTLR;
    uint32 cPMR;
    uint32 cBPR;
    uint32 cIAR;
    uint32 cEOIR;
    uint32 cRPR;
    uint32 cHPPIR;
    uint32 cABPR;
    uint32 cAIAR;
    uint32 cAEOIR;
    uint32 cAHPPIR;
    uint32 cRSVD[52];
    uint32 cIIDR;
} GICCpuInterface_t;
```

Description

Value	Description
cCTLR	**GICC_CTLR**, CPU Interface Control Register
cPMR	**GICC_PMR**, Interrupt Priority Mask Register
cBPR	**GICC_BPR**, Binary Point Register
cIAR	**GICC_IAR**, Interrupt Acknowledge Register
cEOIR	**GICC_EOIR**, End Interrupt Register
cRPR	**GICC_RPR**, Running Priority Register
cHPPIR	**GICC_HPPIR**, Highest Pending Interrupt Register
cABPR	**GICC_ABPR**, Aliased Binary Point Register
cAIAR	**GICC_AIAR**, Aliased Interrupt Acknowledge Register
cAEOIR	**GICC_AEOIR**, Aliased End Interrupt Register
cAHPPIR	**GICC_AHPPIR**, Aliased Highest Pending Interrupt Register
cIIDR	**GICC_IIDR**, CPU Interface Identification Register

Remark

None

7 CALLBACK FUNCTIONS

7.1 GICIsrFunc

This is the 4th parameter type of ***GIC_IntVectSet***. When an interrupt registered by ***GIC_IntVectSet*** occurs, the registered callback function corresponding to the interrupt is called.

Declaration

```
typedef void (*GICIsrFunc)
(
    void * pArg
);
```

Parameters

Value	*Description*
pArg	*[in/out] Callback function argument*

Return

None

Remark

None

8 APIs

8.1 GIC_Init

Initializes the GIC400 block.

Declaration

```
void GIC_Init
(
    void
);
```

Parameters

None

Return

None

Remark

None

8.2 GIC_IntSrcEn

Enables interrupt source ID.

Declaration

```
SALRetCode_t GIC_IntSrcEn
(
    uint32 uiIntId
);
```

Parameters

Value	Description
uiIntId	[in] An index of interrupt source (range of param: 0 ~ GIC_INT_SRC_CNT)

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

8.3 GIC_IntSrcDis

Disables interrupt source ID.

Declaration

```
SALRetCode_t GIC_IntSrcDis
(
    uint32 uiIntId
);
```

Parameters

Value	Description
uiIntId	[in] An index of interrupt source (range of param: 0 ~ GIC_INT_SRC_CNT)

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

8.4 GIC_IntPrioSet

Sets priority of interrupt source ID.

Declaration

```
SALRetCode_t GIC_IntPrioSet
(
    uint32 uiIntId,
    uint32 uiPrio
);
```

Parameters

Value	*Description*
uiIntId	*[in] An index of interrupt source (range of param: 0 ~ GIC_INT_SRC_CNT)*
uiPrio	*[in] Priority (range of param: 0 ~ 15)*

Return

Value	*Description*
SAL_RET_SUCCESS	*Success*
SAL_RET_FAILED	*Failure*

Remark

None

8.5 GIC_IntVectSet

Sets priority of interrupt source id, interrupt type, ISR callback function pointer

Declaration

```
SALRetCode_t GIC_IntVectSet
(
    uint32     uiIntId,
    uint32     uiPrio,
    uint8      ucIntType,
    GICIsrFunc fnIntFunc,
    void *     pIntArg
);
```

Parameters

Value	Description
uiIntId	[in] An index of interrupt source (range of param: 0 ~ GIC_INT_SRC_CNT)
uiPrio	[in] Priority (range of param: 0 ~ 15)
ucIntType	[in] Interrupt type (range of param: refer to Interrupt Type Value)
fnIntFunc	[in] ISR Callback function pointer. Refer to GICIsrFunc.
pIntArg	[in] ISR Callback function parameter pointer

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

8.6 GIC_IntExtSet

Sets external interrupt.

Declaration

```
SALRetCode_t GIC_IntExtSet
(
    uint32 uiIntId,
    uint32 uiGpio
);
```

Parameters

Value	Description
uiIntId	[in] An index of interrupt source (range of param: GIC_EXT0 ~ GIC_EXT9)
uiGpio	[in] GPIO port (Refer to GPIO_GPA(x), GPIO_GPB(x), GPIO_GPC(x), GPIO_GPK(x) macro define of gpio.h)

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

8.7 GIC_IntSGI

Generates SGI.

Declaration

```
SALRetCode_t GIC_IntSGI
(
    uint32 uiIntId
);
```

Parameters

Value	Description
uiIntId	[in] An index of interrupt source (range of param: 0 ~ GIC_INT_SRC_CNT)

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

9 How to use GIC Driver

9.1 Initialize GIC Driver

```
void BSP_PreInit(void)
{
    CLOCK_Init();
    GIC_Init();
    PMU_Init();
}
```

9.2 Example: Using GIC in Device Driver

The following example shows how to use GIC in device driver on GPSB device driver.

```
void GPSB_Init(void)
{
    ……

    /* Register gpsb irq handler */
    (void)GIC_IntVectSet((uint32)GIC_GPSB,
                         (uint32)GIC_PRIORITY_NO_MEAN,
                         (uint8)GIC_INT_TYPE_LEVEL_HIGH,
                         &GPSB_Isr0, NULL_PTR);

    (void)GIC_IntSrcEn(GIC_GPSB);
```

9.3 Example: Using External Interrupt

The following is an example code that uses an external interrupt, and GPIOA00 is used as the door open signal input.

Note: The code below is for example only and is not included in the SDK.

```
static uint32 gIsDoorDet  = 0UL;

static void VehicleSignalExternIntDoor(void *pArg)
{
    uint8 ucDet;

    (void)pArg;
    ucDet      = GPIO_Get(GPIO_GPA(0UL));
    gIsDoorDet = (ucDet != 0UL) ? (1UL) : (0UL);
}

static void VehicleDoorOpen(void)
{
    // DOOR Open
    (void) GPIO_Config(GPIO_GPA(0UL), (GPIO_FUNC(0) | GPIO_INPUT|GPIO_INPUTBUF_EN));
    (void)GIC_IntVectSet(GIC_EXT3, GIC_PRIORITY_NO_MEAN,
                         GIC_INT_TYPE_EDGE_BOTH,
                         VehicleSignalExternIntDoor,
                         NULL_PTR);

    (void)GIC_IntSrcEn(GIC_EXT3);
    (void)GIC_IntExtSet(GIC_EXT3, GPIO_GPA(0UL));
    ……
}
```

10 REFERENCES

[1] TCC70xx Full Specification

[2] Contact Telechips for more details: sales@telechips.com

11 REVISION HISTORY

Rev. 0.10: 2021-09-03

- Preliminary version release

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.
No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.
This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.
Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:
"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit http://mp3licensing.com".

For customers who use other firmware of mp3:
"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit http://mp3licensing.com".

For customers who use Digital Wave DRA solution:
"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:
"This product made under license to certain U.S. patents and/or foreign counterparts."
"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:
"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:
"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:
"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."